DuocUC | ONLINE



Guía de aprendizaje

Implementando microservicios y seguridad en la nube con Spring Cloud

Exp 3 – Semana 6

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villarroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....3
Conceptos relevantes....4
Preguntas activadoras....4
Actividad....4
¿Recuerdas los microservicios?....6
Comunicación en los microservicios....7
Tolerancia a fallos....8
Seguridad....10
Levantar un ecosistema Spring Cloud....13
Config Server....14
Microservicios consumidores....15
Service Discovery....17
Tolerancia a Fallos....18
Security....19
Lectura complementaria....23
Cierre de la semana....25
Referencias....26
Lecturas de la semana....27
Apuntes....28

Introducción a la semana



En esta semana, te adentrarás en el desarrollo de componentes backend avanzados enfocados en microservicios y resiliencia, utilizando tecnologías de la nube como Spring Cloud y herramientas clave como Resilience4j. El objetivo es implementar sistemas que no solo faciliten la integración y configuración eficiente entre servicios, sino que también sean resistentes a fallos, siguiendo las mejores prácticas de la industria.

Explorarás cómo construir y configurar microservicios con Spring Cloud, aprendiendo a gestionar servidores de configuración centralizados, el descubrimiento de servicios y la integración a través de un gateway. Además, te familiarizarás con la implementación de mecanismos de seguridad y tolerancia a fallos, utilizando Resilience4j para garantizar que el sistema pueda enfrentar fallos de manera controlada y mantener su funcionamiento.

A través de estos aprendizajes, darás un paso más hacia el desarrollo de sistemas backend robustos, distribuidos y escalables, capaces de operar en entornos complejos y de alta disponibilidad. Al finalizar, serás capaz de construir soluciones cloud que garanticen tanto la correcta interacción entre servicios como la resiliencia ante posibles interrupciones.

Resultado de aprendizaje

El estudiante será capaz de:

RA3. Desarrolla componentes backend avanzados utilizando tecnologías cloud que permitan disponibilizar sistemas resistentes a fallos, de acuerdo a las buenas prácticas de la industria.

Indicador de logro:

IL6. Implementa microservicios utilizando Spring Cloud, para configurar servidores y garantizar la correcta integración entre servicios.

IL7. Configura mecanismos de seguridad y tolerancia a fallos utilizando Resilience4j, garantizando la resiliencia del sistema ante fallos.

Conceptos relevantes

Microservicios	Resiliencia	Spring Cloud
Config Server	Eureka	Circuit Breaker

Preguntas activadoras

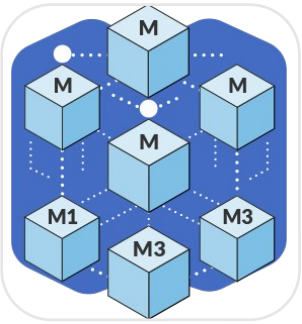
- ¿Por qué es útil dividir un sistema en microservicios?
- ¿Cómo puede un sistema mantenerse funcionando incluso cuando un servicio falla?
- ¿Qué crees que aporta Spring Cloud a la construcción de microservicios?

Actividad

Descripción de la actividad

En esta sexta semana, desarrollarás una actividad formativa grupal, llamada "Implementando microservicios y seguridad en la nube con Spring Cloud", donde deberás generar una entrega por encargo, focalizándote en el desarrollo de microservicios resilientes utilizando Spring Cloud; tendrás que configurar servidores para garantizar una integración eficiente entre los diferentes servicios que componen una arquitectura distribuida en la nube. Además, implementarás mecanismos de seguridad en la nube con Spring Cloud Security, asegurando la protección de la infraestructura distribuida.

¿Recuerdas los microservicios?



Recordemos que los microservicios son un estilo arquitectónico que divide un sistema grande en pequeñas aplicaciones independientes, cada una responsable de una funcionalidad específica. Estas aplicaciones se comunican entre sí mediante APIs. A diferencia de un sistema monolítico, donde todo está en una sola aplicación, los microservicios ofrecen ventajas como:

- Escalabilidad: puedes escalar solo los servicios que lo necesiten.
- Despliegue independiente: actualizas o despliegas un servicio sin afectar a los demás.
- Mantenimiento más sencillo: el código de cada servicio es más pequeño y manejable.

Cuando trabajas con microservicios, la complejidad de administración puede ser exponencial. Imagina administrar 100 microservicios o más, ¿crees que es sencillo el despliegue? Ahí es donde Spring Cloud juega un papel esencial.

Link de Interés

Si deseas repasar con mayor profundidad, puedes revisar el siguiente link, donde responderás preguntas como: ¿qué son los microservicios?, o ¿por qué crear microservicios?

https://spring.io/microservices

Comunicación en los microservicios

Cuando trabajas con microservicios, estás construyendo un sistema dividido en varias partes pequeñas, cada una encargada de una tarea específica. Estas partes (o servicios) necesitan comunicarse entre sí para que todo el sistema funcione como un todo. Aquí es donde Spring Cloud entra en juego: te proporciona herramientas diseñadas para facilitar esta comunicación y gestión.

Algunas de las herramientas más importantes que usarás son:

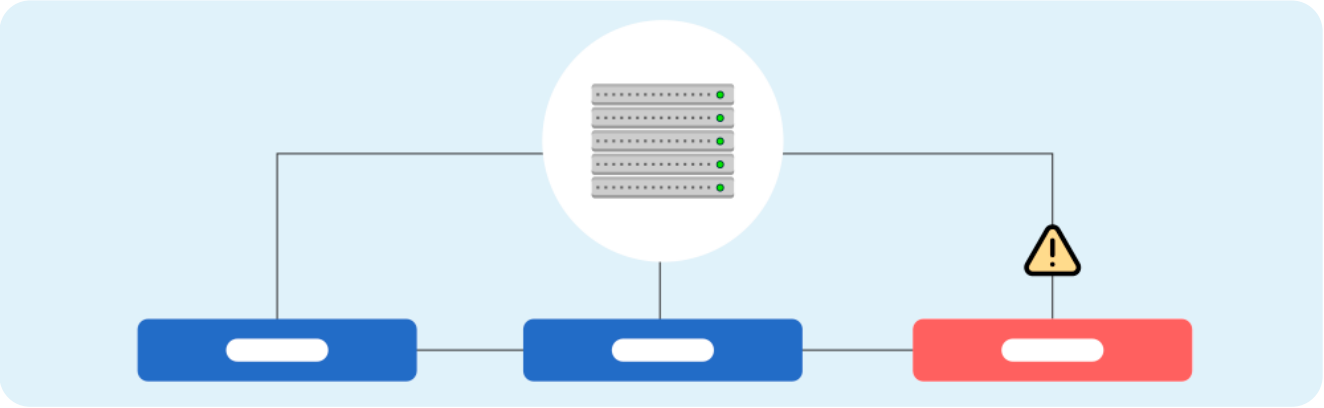
- **Config Server** (Servidor de Configuración):
  Imagina que cada microservicio necesita ciertas instrucciones para funcionar, como el puerto que usa o la conexión a una base de datos. En lugar de que cada servicio tenga sus configuraciones guardadas por separado, el Config Server actúa como un lugar central donde puedes guardar todas estas configuraciones. Así, si necesitas cambiar algo, solo lo haces una vez en el Config Server, y todos los servicios lo recibirán automáticamente. Esto no solo ahorra tiempo, sino que también evita errores de configuración.
- **Eureka** (Service Discovery o Descubrimiento de Servicios):
  Los microservicios necesitan saber cómo encontrar a otros servicios en el sistema, por ejemplo, si un servicio quiere enviar datos a otro, primero necesita conocer su dirección. Con Eureka, no tienes que configurar manualmente estas direcciones, porque Eureka permite que los servicios "se registren" y encuentren automáticamente a los demás. Esto hace que la comunicación entre servicios sea mucho más sencilla y flexible.
- **Gateway** (Puerta de enlace):
  Piensa en el Gateway como un "portero" para tu sistema. Todas las solicitudes externas (por ejemplo, de un usuario que usa tu aplicación) pasan primero por el Gateway, que decide a qué microservicio enviar esa solicitud. Además, el Gateway puede encargarse de verificar si alguien tiene permiso para acceder al sistema (autenticación), manejar las reglas de acceso (autorización) y asegurarse de que todo funcione de manera segura.

Importante

Estas herramientas no solo hacen que sea más fácil trabajar con microservicios, sino que también te permiten enfocarte en lo más importante ***"Diseñar y construir las funcionalidades específicas que cada servicio debe cumplir"***. Spring Cloud se encarga del resto, ayudándote a conectar, gestionar y asegurar tus servicios de manera eficiente.

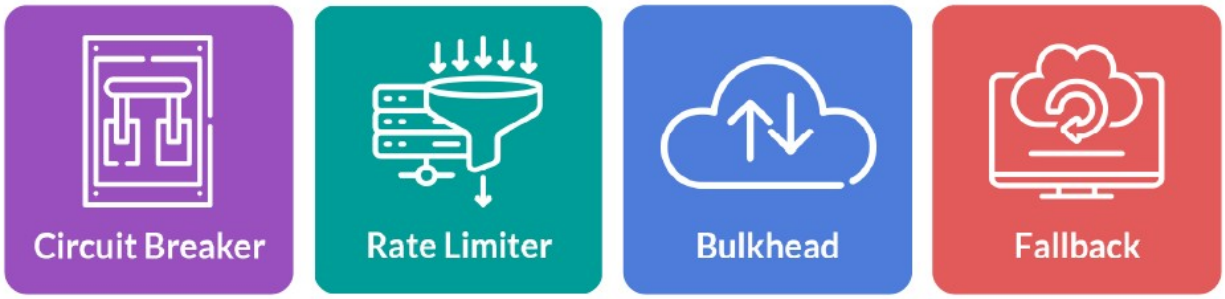
Tolerancia a fallos

Cuando trabajas con microservicios, estás construyendo un sistema que depende de muchas partes pequeñas trabajando juntas. Sin embargo, en cualquier momento, algo puede salir mal. Un servicio podría dejar de responder, la red podría volverse lenta o el sistema podría recibir más solicitudes de las que puede manejar. Si no preparas tu sistema para enfrentar estos problemas, un fallo pequeño podría convertirse en un problema grande que afecte a todo el sistema.



Para evitar que esto suceda, necesitas implementar tolerancia a fallos, que es como darle a tu sistema la capacidad de "resistir golpes" y seguir funcionando. Herramientas como **Resilience4j**, te ayudan a lograrlo.

Estas herramientas ofrecen funcionalidades clave como:



- **Circuit Breaker** (Interruptor de circuito):

Piensa en esto como un "fusible" para tu sistema. Si un servicio tarda mucho en responder o no está disponible, el Circuit Breaker corta la conexión con ese servicio. Esto evita que los problemas de un servicio afecten al resto del sistema.

Por ejemplo, si un servicio de pagos no responde, los otros servicios (como el carrito de compras) seguirán funcionando sin quedarse bloqueados esperando una respuesta.

- **Rate Limiter** (Limitador de velocidad):

Se utiliza para controlar la velocidad de las solicitudes entrantes a un servicio. Cuando se alcanza el límite de velocidad, las solicitudes entrantes se rechazan o se retrasan hasta que la velocidad cae por debajo del límite. Se puede configurar estableciendo el número máximo de solicitudes en un tiempo determinado.

- **Bulkhead** (Mamparo):

Permite limitar la cantidad de llamadas simultáneas a los servicios, evitando la sobrecarga del sistema.

- **Fallback** (Respuesta alternativa):

Cuando un servicio falla, puedes configurar una respuesta de respaldo (fallback) para que el sistema no se quede sin responder. Por ejemplo, si un servicio de recomendaciones no está disponible, en lugar de mostrar un error, podrías mostrar un mensaje genérico como "Intenta más tarde" o incluso datos predeterminados. Esto no solo mejora la experiencia del usuario, sino que también evita que los fallos se propaguen.

Además, en tu vocabulario deberás agregar un atributo de calidad clave, ***"Resiliencia"***. La resiliencia es la capacidad de un sistema para seguir funcionando incluso cuando algo falla. En un entorno de microservicios, esto es crucial, ya que garantiza que las fallas en una parte del sistema no arruinen la experiencia para los usuarios o interrumpan completamente la operación del sistema.

Preparar tu sistema para manejar fallos no solo te ayuda a evitar problemas mayores, sino que también mejora la confiabilidad y la percepción de calidad de tus aplicaciones. Al usar herramientas como Circuit Breakers y Fallbacks, puedes crear sistemas robustos que estén listos para enfrentar desafíos en el mundo real.

Seguridad

Cuando trabajas con sistemas distribuidos, como los que utilizan microservicios, tienes múltiples servicios que necesitan interactuar entre sí, además de permitir el acceso de usuarios externos. Cada uno de estos puntos de entrada representa un posible riesgo de seguridad. Por eso, proteger el sistema y los datos es esencial. Y, de no tomar medidas adecuadas, podrías exponer información sensible o permitir accesos no autorizados.

Aquí tienes algunas prácticas básicas que puedes implementar para asegurar tu sistema:

1. **Autenticación y autorización**

Esto garantiza que solo los usuarios y servicios correctos puedan acceder a ciertas partes de tu sistema.



Autenticación



Autorización

- **Autenticación**: comprueba quién eres (por ejemplo, usando una contraseña). Existen diferentes mecanismos para validar “quién eres”:
  - Autenticación Básica
  - JWT (JSON Web Token)
  - MFA (Multi factor auth)
  - Biometría
  - Certificados
- **Autorización**: determina qué puedes hacer (por ejemplo, acceder a datos específicos). Utiliza la sesión del usuario adquirida a través de los mecanismos de autenticación.

2. **Identidad delegada a un tercero**
   En lugar de que tu aplicación gestione directamente las credenciales de los usuarios, delegas esta responsabilidad a un servicio externo especializado
   - **SSO:** permite a los usuarios iniciar sesión una sola vez y acceder a múltiples aplicaciones sin necesidad de autenticarse nuevamente, mejorando la experiencia del usuario
   - **SAML:** es un estándar que delega la autenticación en un proveedor de identidad, enviando un "ticket" de seguridad que valida al usuario en diferentes aplicaciones.

- **LDAP:** es un protocolo que centraliza la gestión de usuarios y contraseñas en un directorio, permitiendo que múltiples sistemas consulten esa información de forma eficiente.
- **OAuth 2.0:** es un protocolo que permite a los usuarios otorgar acceso limitado a sus recursos a una aplicación, sin necesidad de compartir sus credenciales.
- **OIDC**: es una extensión de OAuth 2.0 que autentica al usuario y proporciona información básica sobre su identidad a las aplicaciones.

3. **Comunicación segura**
   Los microservicios se comunican constantemente entre sí, y debes asegurarte de que esta comunicación no pueda ser interceptada o manipulada.
   - Usa **HTTPS** para encriptar las comunicaciones. Esto asegura que los datos que viajan entre servicios no puedan ser leídos por atacantes.
   - Considera usar certificados mutuos (**mTLS**) para verificar tanto al emisor como al receptor de los mensajes.

Implementando estas prácticas, puedes evitar accesos no autorizados, proteger los datos sensibles que se transmiten entre servicios y garantizar que cada parte del sistema solo reciba las solicitudes adecuadas. Estas medidas no solo hacen que tu sistema sea más seguro, sino que también generan confianza en los usuarios y en las personas que dependen de él.

Asegurar tu sistema desde el inicio te permitirá construir una base sólida para enfrentar las amenazas del mundo real. Con autenticación, comunicación segura y control centralizado, estarás listo para proteger tu sistema frente a posibles ataques o vulnerabilidades.

Levantar un ecosistema Spring Cloud

Ahora que hemos recordado los aspectos clave sobre los microservicios, estás listo para comenzar a explorar cómo levantar un ecosistema avanzado utilizando Spring Cloud. Esta herramienta te permite implementar funcionalidades esenciales como configuración centralizada, descubrimiento de servicios, balanceo de carga, gateways y tolerancia a fallos, facilitando la creación de un sistema distribuido escalable y resiliente.

Links de Interés

En esta sección, revisaremos ejemplos prácticos que te ayudarán a comprender cómo integrar cada componente de Spring Cloud para enfrentar los desafíos típicos de las arquitecturas modernas de microservicios.

Revisa cada uno de los siguientes links donde podrás revisar los ejemplos detallados y oficiales de Spring Cloud:

- Ejemplo de Config Server: https://github.com/spring-cloud-samples/configserver
- Ejemplo de microservicio: https://github.com/spring-cloud-samples/customers-stores
- Ejemplo Eureka Server: https://github.com/spring-guides/gs-service-registration-and-discovery
- Ejemplo de Gateway: https://github.com/spring-guides/gs-gateway
- Ejemplo de Circuit Breaker: https://github.com/spring-guides/gs-cloud-circuit-breaker
- Ejemplo de Seguridad: https://github.com/spring-guides/gs-securing-web
- Ejemplo de LDAP*: https://github.com/spring-guides/gs-authenticating-ldap



Importante

Todos los ejemplos anteriores, los puedes crear tú mismo desde 0 también.

***El ejemplo LDAP solo podrá funcionar si tienes algún servidor LDAP al cual conectarte. En las empresas es super común encontrar este tipo de proyectos, así que te será de utilidad revisar una implementación de referencia.**

Config Server

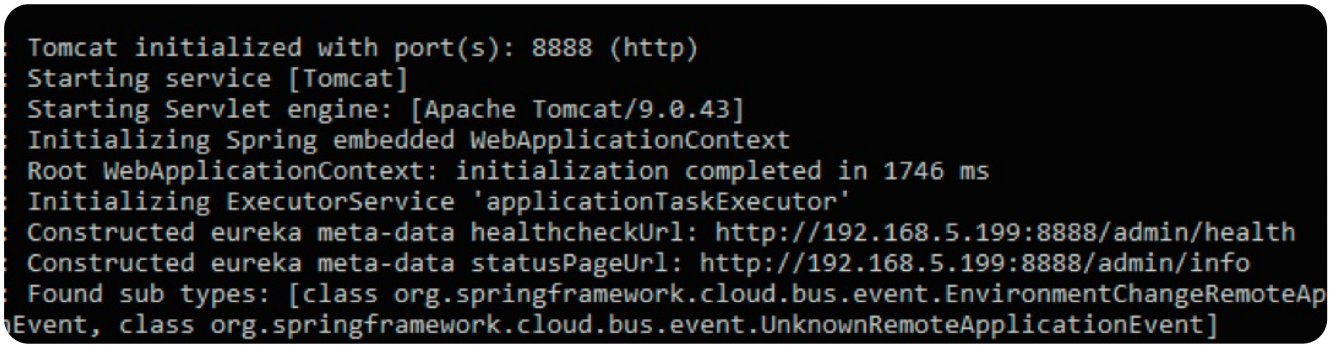
Para levantar un config server, puedes utilizar ***Ejemplo de Config Server***. Eso sí, debes considerar:

- Revisar tu versión de Java, ya que el ejemplo solo corre con versión 1.7 y 1.8.
- Si quieres usar cache, debes levantar con docker-compose la configuración del archivo docker-compose.yml

Cuando la aplicación config server se ejecuta correctamente, te entregará la URL a la cual podrás acceder para obtener las configuraciones referenciadas.

Figura 1

Config Server ejecutándose correctamente



Nota. La imagen muestra la aplicación config server ejecutándose correctamente. Microsoft (2006). Power Shell. (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

Microservicios consumidores

Para utilizar Config Server puedes utilizar ***Ejemplo de microservicio*** o podrás tomar uno de tus microservicios desarrollados en Backend I o II y configurar la nueva integración.

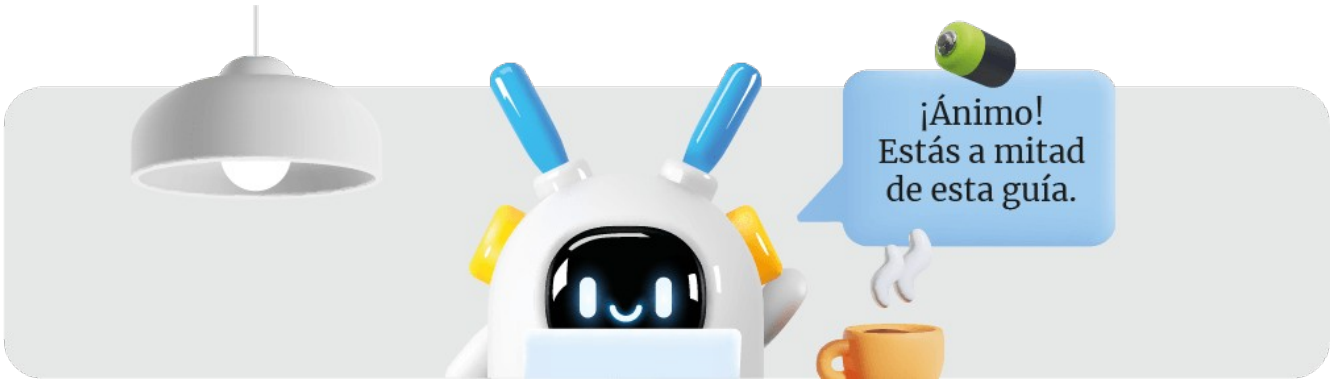
Figura 2

Dependencia de config server en el microservicio.

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

Nota. La imagen muestra la dependencia necesaria para que un microservicio pueda conectarse a un config server. Power Shell. (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

Luego de agregar la dependencia, deberás crear y agregar parámetros a un archivo llamado **bootstrap.yml** que Spring Cloud utilizará para ejecutar ciertos comportamientos en tu aplicación.



Links de Interés

Para saber más sobre el contexto de Spring Cloud con Bootstrap, visita el siguiente link: https://cloud.spring.io/spring-cloud-commons/multi/multi__spring_cloud_context_application_context_services.html

Figura 3

Configuración de microservicio en archivo bootstrap.yml



Nota. La imagen muestra la dependencia de una configuración básica de Spring Cloud. Microsoft. (2024). Visual Studio Code. (1.95). [Software]. https://code.visualstudio.com

La dependencia de una configuración básica de Spring Ccoud te permitirá consumir config server desde el microservicio. Para configurar un application.yml, asegúrate de seguir las siguientes reglas:

- Todos los archivos de configuración pueden quedar alojados en un S3 (aws) o github (repositorio). Mira este el siguiente ejemplo: https://github.com/spring-cloud-samples/config-repo

- El archivo a crear deberá tener el mismo nombre de la propiedad **spring.application.name**, ya que, con ello, el microservicio realizará la búsqueda en config server.

Se te invita a configurar algún microservicio que ya poseas y delegues la configuración de entorno a un config server.

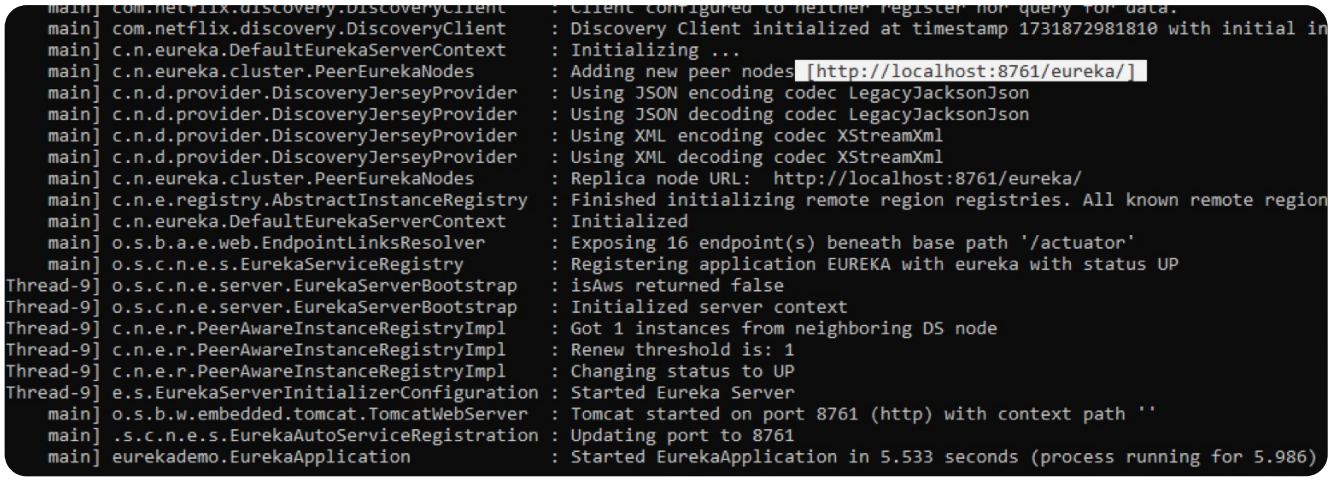
Service Discovery

Para poder utilizar un servidor encargado del descubrimiento de servicios, utiliza el proyecto ***Ejemplo de Eureka Server*** y considera:

- Revisar tu versión de Java, ya que el ejemplo solo corre con versión 17.
- Una vez revisada la versión, tu servidor debería ejecutarse como muestra la Figura 4.

Figura 4

Eureka server en funcionamiento



Nota. La imagen muestra Eureka server operando correctamente. Microsoft. (2024). Visual Studio Code. (1.95). [Software]. https://code.visualstudio.com

Microservicios consumidores

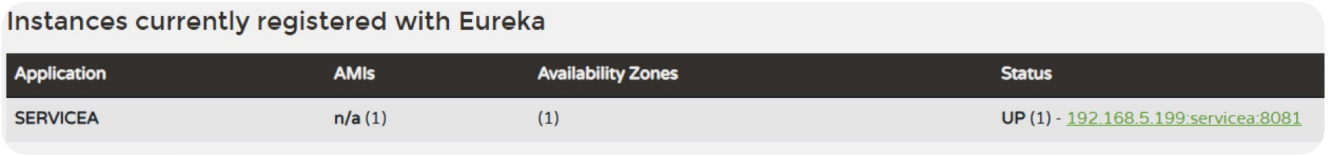
En los servicios consumidores, deberás validar que posean:

- La dependencia spring-cloud-starter-netflix-eureka-client en el archivo pom.xml
- Luego de ello, puedes revisar en el proyecto de Eureka server -> ServiceB como se realiza una petición HTTP GET a un servicio registrado en Eureka.

De esta manera, ya no tendrás que preocuparte de configurar indeterminadas URL en tu archivo de configuración.

Figura 5

Registro exitoso de Servicio en Eureka



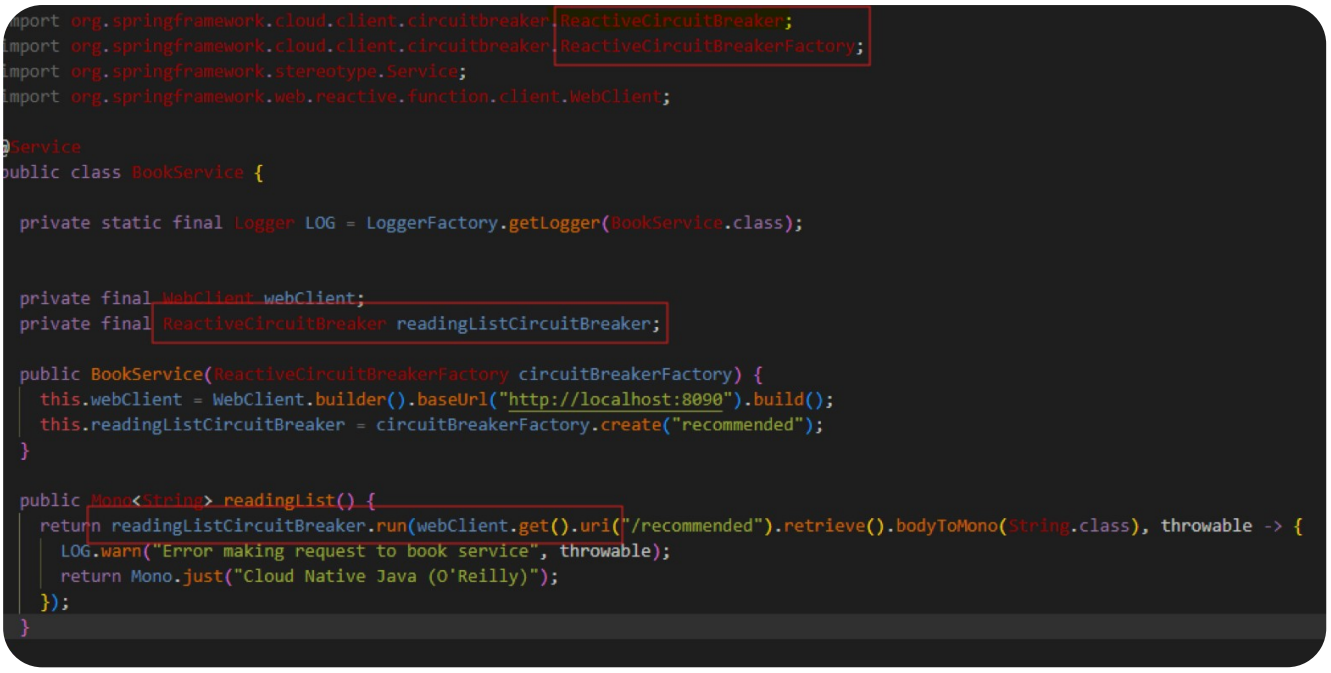
Nota. La imagen muestra un registro correcto de un microservicio llamado SERVICEA en Eureka server. Spring Cloud Eureka Server. (s.f.). Eureka server. [Software]. https://cloud.spring.io/spring-cloud-netflix/reference/html/

Tolerancia a Fallos

Para implementar Circuit Breaker, usa el proyecto ***Ejemplo de Cricuit Breaker***. Lo más importante es lo que aparece marcado en la figura 6, ya que es el consumidor de un servicio externo quien debe asegurar el mecanismo. En este caso el BookService realiza una consulta a otro servicio y para asegurar la resiliencia de BookService, se utiliza el patrón Circuit Breaker más el Fallback (Respuesta alternativa).

Figura 6

Implementación de Circuit Breaker



```
import org.springframework.cloud.client.circuitbreaker.ReactiveCircuitBreaker;
import org.springframework.cloud.client.circuitbreaker.ReactiveCircuitBreakerFactory;
import org.springframework.stereotype.Service;
import org.springframework.web.reactive.function.client.WebClient;

@Service
public class BookService {

  private static final Logger LOG = LoggerFactory.getLogger(BookService.class);


  private final WebClient webClient;
  private final ReactiveCircuitBreaker readingListCircuitBreaker;

  public BookService(ReactiveCircuitBreakerFactory circuitBreakerFactory) {
    this.webClient = WebClient.builder().baseUrl("http://localhost:8090").build();
    this.readingListCircuitBreaker = circuitBreakerFactory.create("recommended");
  }

  public Mono<String> readingList() {
    return readingListCircuitBreaker.run(webClient.get().uri("/recommended").retrieve().bodyToMono(String.class), throwable -> {
      LOG.warn("Error making request to book service", throwable);
      return Mono.just("Cloud Native Java (O'Reilly)");
    });
  }
```

Nota. La imagen muestra el uso de la clase y la factoria “ReactiveCircuitBreaker”. Microsoft. (2024). Visual Studio Code. (1.95). [Software]. https://code.visualstudio.com./

Con el uso de la clase y la factoría “ReactiveCircuitBreaker”, el sistema asegura un mecanismo anti-fallos si es que el servicio /recommend llega a presentar algún problema de disponibilidad.

Security

Para revisar el ejemplo de seguridad en Spring Cloud, puedes usar ***Ejemplo de Seguridad*** para poder ver sus configuraciones a nivel de código.

Cosas a tener en cuenta:

- Credenciales user/password
- El método de autenticación cambio de formLogin a httpBasic

La Figura 7 muestra una configuración básica de seguridad en la aplicación Spring Boot usando Spring Security. Define un conjunto de reglas para manejar quién puede acceder a las diferentes rutas de tu aplicación, por ejemplo, las rutas "/" y "/home" están abiertas para todos, pero cualquier otra ruta requerirá que el usuario inicie sesión.

También establece un inicio de sesión basado en formulario, usando la página "/login" como el punto de entrada para autenticar a los usuarios, y permite que cualquier persona pueda cerrar sesión. Además, se crea un usuario en memoria con el nombre de usuario user, la contraseña password y el rol USER, lo que significa que este usuario tiene permisos básicos en el sistema. Esto te permite probar cómo funcionan la autenticación y autorización en una aplicación, aunque debes saber que en proyectos reales se usarían contraseñas codificadas de forma segura y bases de datos para almacenar los usuarios, en lugar de guardar todo en memoria. Este ejemplo te ayuda a entender cómo proteger tus rutas y controlar el acceso de los usuarios de manera sencilla.

Figura 7

Habilitación de seguridad en un microservicio



```java
@Configuration
@EnableWebSecurity
public class WebSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests((requests) -> requests
                .requestMatchers("/", "/home").permitAll()
                .anyRequest().authenticated()
            )
            .httpBasic()
            .and()
            .logout((logout) -> logout.permitAll());

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user =
            User.withDefaultPasswordEncoder()
                .username("user")
                .password("password")
                .roles("USER")
                .build();

        return new InMemoryUserDetailsManager(user);
    }
}
```

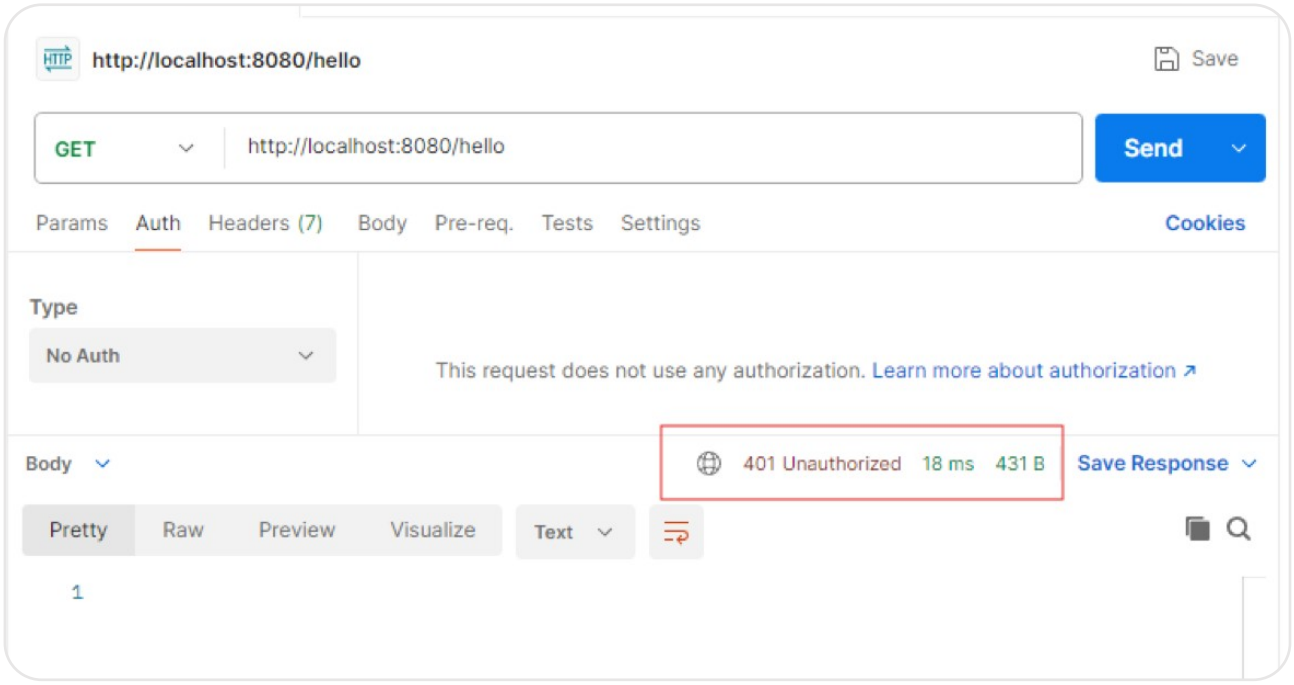
Nota. La imagen muestra la clase WebSecurityConfig. Microsoft. (2024). Visual Studio Code. (1.95). [Software]. https://code.visualstudio.com

La clase WebSecurityConfig será usada para la gestión de permisos en la aplicación. También se destaca que el ejemplo original ocupa un formulario web, y en este ejemplo fue cambiado a httpBasic para poder realizar interacciones con Postman.

Con un protocolo de seguridad, ahora tus peticiones deberán proveer la información necesaria para poder validarse contra la configuración de seguridad. En el siguiente caso, cuando no se envían las credenciales correctas al servicio, este retornará un 401 (no autorizado).

Figura 8

Fallo de autenticación

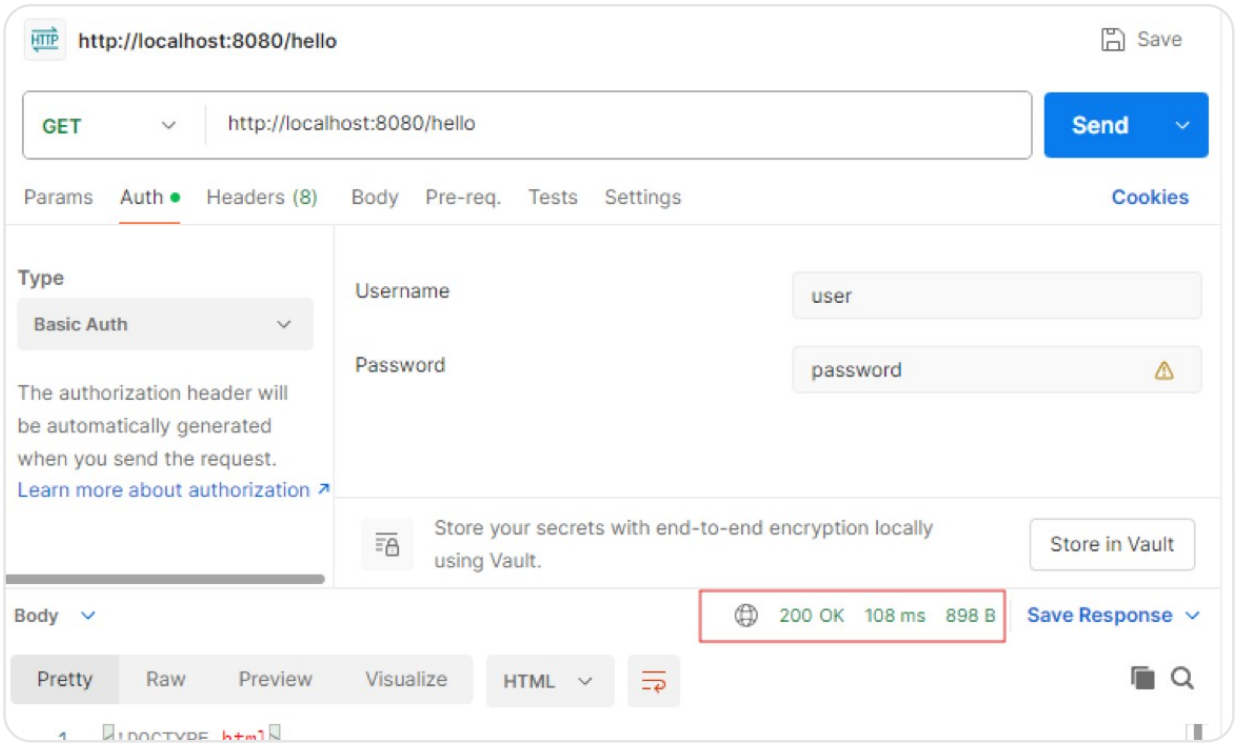


Nota. La imagen muestra una validación incorrecta, con mensaje de retorno de 401 (no autorizado). Asthana, A. Sobti, A. & Kane, A. (2012). Postman. (11.19) [Software]. https://www.postman.com/

Una vez que le envíes los datos en Auth → Basic Auth, el servicio podrá validarse correctamente.

Figura 9

Autenticación correcta



Nota. La imagen muestra la validación correcta. Asthana, A. Sobti, A. & Kane, A. (2012). Postman. (11.19) [Software]. https://www.postman.com

Lectura complementaria

Puedes encontrar más información sobre Spring Cloud en los siguientes artículos:
https://spring.io/projects/spring-cloud#overview

Cierre de la semana



¡Has completado una semana enfocada en desarrollar y asegurar sistemas Backend avanzados utilizando la tecnología de Spring Cloud! A lo largo de estos días, exploraste cómo construir microservicios resilientes con Spring Cloud, aprendiendo a configurar servidores de configuración centralizados, implementar descubrimiento de servicios con Eureka y gestionar la comunicación a través de un API Gateway.

Además, profundizaste en la implementación de mecanismos de seguridad y tolerancia a fallos, utilizando herramientas como Resilience4j. Estas prácticas te permitieron asegurar que los sistemas sean resistentes a interrupciones, manteniendo la estabilidad y confianza de los usuarios incluso en escenarios adversos. También comprendiste la importancia de proteger la infraestructura distribuida mediante autenticación, autorización y comunicación segura, con conceptos como JWT, OAuth 2.0 y mTLS.

Ahora, estás mejor preparado para diseñar arquitecturas distribuidas que no solo sean eficientes y escalables, sino también seguras y resilientes. Con este conocimiento, estás listo para enfrentar los retos del desarrollo cloud moderno. ¡Sigue adelante, perfeccionando tus habilidades y construyendo soluciones robustas para entornos distribuidos!

Referencias

- Amán, M. (s,f.). Boosting Application Resilience: Your Guide to Resilience4j. Mind Bowser. https://www.mindbowser.com/resilience4j-guide-for-app-resilience/
- Spring. (s.f.). Spring Cloud Circuit Breaker. Spring. https://spring.io/projects/spring-cloud-circuitbreaker
- Spring. (s.f.). Spring Cloud Config. Spring. https://spring.io/projects/spring-cloud-config#overview
- Spring. (s.f.). Spring Cloud Gateway. Spring. https://spring.io/projects/spring-cloud-gateway
- Spring. (s.f.). Spring Cloud Netflix. Spring. https://spring.io/projects/spring-cloud-netflix
- Spring. (s.f.). Spring Cloud Security. Spring. https://spring.io/projects/spring-cloud-security

Lecturas de la semana

- AWS (s.f.). AWS: Circuit Breaker. https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html
- Microsoft (s.f.). Patrón Circuit Breaker. https://learn.microsoft.com/es-es/azure/architecture/patterns/circuit-breaker
- AWS (s.f.). Patrón API Gateway. https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/modernization-integrating-microservices/api-gateway-pattern.html
- Microsoft (s.f.) Definición de autenticación y autorización. https://www.microsoft.com/es-cl/security/business/security-101/what-is-authentication

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.